

Datové struktury a algoritmy

Přednáška 2, poznámky

Jan Voráček

VŠPJ Jihlava, voracek@vspj.cz

Úvodní informace

- Administrativa
 - Komentáře k Průvodci studiem nebo další dotazy k přednáškám?
 - Test 1: Základní orientace v kurzu DSA absolvovali včas pouze 4 studenti (bonus)
 - Od 19.10. budou přednášky DSA v učebně P3
- Prezentace: včas odevzdány pouze 2 prezentace ze 4 (**penalizace**)
 - Algoritmy a způsoby jejich textové reprezentace
 - Základní datové typy jazyka C++
 - Abstraktní datové typy (ADT) a způsoby jejich textové reprezentace
- Témata
 - Abstraktní datové typy
 - Programovací jazyky
 - Pole a spojový seznam: vlastnosti a implementace
 - První bonusový on-line přednáškový test v Moodle

Přednášky doc. Hudce, část 1 (úvod)

Algoritmy v C++

- Cíl předmětu
 - zdokonalit se v návrhu a implementaci abstraktních datových typů
 - seznámit se s efektivními algoritmy operací nad ADT a naučit se sestavovat algoritmy řešení složitějších problémů
 - naučit se využívat knihovnu STL (Standard Template Library)
- Organizace předmětu
 - 2 hod. přednášek
 - 2 hod. cvičení v počítačové učebně
 - 1 test na cvičení
 - domácí úlohy
 - semestrální práce
 - zkouška (2 příklady na papíře + doplňující otázka)
- Studijní materiály a další informace budou postupně zveřejňovány na
S:UCITELE/Hudec/AVC

Základní datové struktury - opakování

Pole

- Pole lze zavést jako abstraktní datový typ. Zatím se spokojíme s neformální specifikací pole:
 - Pole je tvořeno **pevným počtem prvků stejného typu**, které jsou v paměti uloženy jeden za druhým
 - Počet prvků pole je dán při vytvoření pole a nelze jej měnit
 - Přístup k prvkům pole pomocí **indexu**, což jsou celá čísla od 0 do $N-1$, kde N je počet prvků pole (v některých jazycích může být dolní mez indexu nenulová)

`int a[N];` `a`

--	--	--	--	--

 `a[0]` `a[1]` `a[2]` `...` `a[N-1]`

- Přístup k prvku pole je operace s **konstantní složitostí**, tj. nezávisí ani na počtu prvků pole, ani na indexu

Pole

- Ve většině programovacích jazyků je pole základní strukturovaný datový typ. Zaměříme se na jazyk C++.
 - V C/C++ musí být počet prvků N deklarovaného pole dán konstantou
Některé implementace (např. GCC) dovolují, aby počet prvků pole deklarovaného ve funkci byl dán proměnnou (zakázat to lze parametrem *-pedantic*)
 - Pole, jehož počet prvků má být dán proměnnou, můžeme vytvořit dynamicky
 - n -rozměrné pole chápeme jako pole $n-1$ rozměrných polí

Pole – příklad použití

Eratosthenovo sito

- Algoritmus, který najde všechna prvočísla z intervalu 2 až N :
 1. Vytvoříme množinu obsahující všechna přirozená čísla od 2 do N .
 2. Z množiny vypustíme všechny násobky čísla 2.
 3. Najdeme nejbližší číslo k tomu, jehož násobky jsme v předchozím kroku vypustili, a vypustíme všechny násobky tohoto čísla.
 4. Opakujeme krok 3, dokud číslo, jehož násobky jsme vypustili, není větší než odmocnina z N .
 5. Čísla, která v množině zůstanou, jsou hledaná prvočísla.
- Pro reprezentaci množiny čísel použijeme **pole a prvků typu `bool`**
 - prvek `a[x]` bude udávat, zda číslo x v množině je (`true`) nebo není (`false`)
- Řešení ve třech programech
 - `p1/sito1.cpp` velikost pole dána konstantou
 - `p1/sito2.cpp` velikost pole dána proměnnou
 - `p1/sito3.cpp` pole vytvořeno dynamicky
- Nejprve ale simulace algoritmu

Eratosthenovo síto

- Simulace algoritmu pro $N = 30$

1. Vytvoření počáteční množiny

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

2. Vypuštění násobků 2 (4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30)

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

3. Nejbližší číslo je 3, vypuštění násobků 3 (6, 9, 12, 15, 18, 21, 24, 27)

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

3. Nejbližší číslo je 5, vypuštění násobků 5 (10, 15, 20, 25)

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

Výsledek: prvočísla do 30 jsou 2, 3, 5, 7, 11, 13, 17, 19, 23, 29

Eratosthenovo síto

- Simulaci algoritmu předvedeme ještě jednou, přičemž budeme zobrazovat též obsah pole realizujícího množinu (1=true, 0=false)

1. Vytvoření počáteční množiny

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30

2. Vypuštění násobků 2 (4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30)

1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30

3. Nejbližší číslo je 3, vypuštění násobků 3 (6, 9, 12, 15, 18, 21, 24, 27)

1	1	0	1	0	1	0	0	0	1	0	1	0	0	0	1	0	1	0	0	0	1	0	1	0	0	0	1	0
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30

3. Nejbližší číslo je 5, vypuštění násobků 5 (10, 15, 20, 25)

1	1	0	1	0	1	0	0	0	1	0	1	0	0	0	1	0	1	0	0	0	1	0	0	0	0	0	1	0
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30

Výsledek: prvočísla do 30 jsou 2, 3, 5, 7, 11, 13, 17, 19, 23, 29

Eratosthenovo síto

- Eratosthenovo síto 1 – velikost pole dána konstantou

```
/* sito1.cpp */
...

const int N = 1000;

int main() {
    int i, j;
    bool a[N+1];          /* inicializace pole a */
    for (int i=2; i<=N; i++) a[i] = true;
    int p = 2, pmax = (int)sqrt(N);
    do {
        // vypusteni vsech nasobku cisla p
        for (int i=p+p; i<=N; i+=p) a[i]=false;
        // hledani nejblizsiho cisla k p
        do {
            p++;
        } while (!a[p]);
    } while (p<=pmax);
    for (int i=2; i<=N; i++) /* tisk pvocisel */
        if (a[i]) cout << i << ' ';
    cout << endl;
    system("PAUSE");
    return 0;
}
```

Eratosthenovo síto

- Eratosthenovo síto 2 – velikost pole dána proměnnou, pole vytvořeno dynamicky

```
/* sito2.cpp */  
...  
int main() {  
    int i, j, N;  
    cout << "zadej N: ";  
    cin >> N;  
    bool *a = new bool[N+1];  
    ...  
}
```

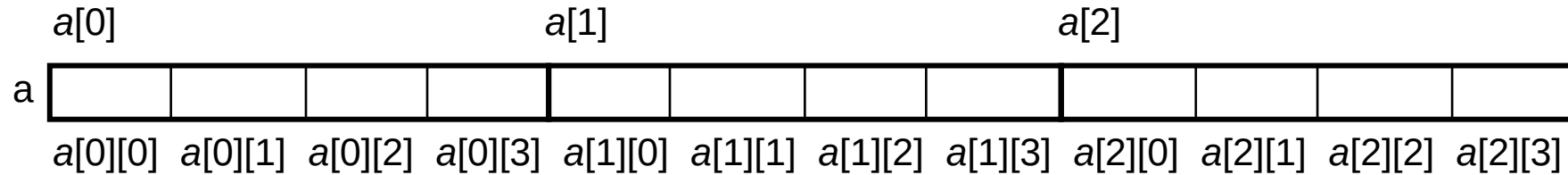
- Eratosthenovo síto 3 – velikost pole dána proměnnou, pole vytvořeno deklarací **Jen v některých implementacích**, např. GCC bez parametru *-pedantic*

```
/* sito3.cpp */  
...  
int main() {  
    int i, j, N;  
    cout << "zadej N: ";  
    cin >> N;  
    bool a[N+1];  
    ...  
}
```

Dvozměrné pole

- Dvozměrné pole prvků typu T je pole, jehož prvky jsou pole prvků typu T

```
const int M = 3, N = 4;  
int a[M][N];
```



- V ANSI C/C++ musí M i N být konstanty, GCC umožňuje i proměnné
- Příklad: `p1/matice1.cpp` (použijte GCC s/bez parametru `-pedantic`)

Dvojrozměrné pole a ukazatele

- Znovu se podívejme na pole zavedené deklarací

```
const int M = 3, N = 4;  
int a[M][N];
```

- Jakého typu je?

`a[i][j]`

int

`a[i]`

*int **

ukazatel na *int*

`a`

int ()[4]*

ukazatel na pole 4 prvků typu *int*

- Můžeme tedy deklarovat a inicializovat následující proměnné

```
int (*p)[4] = a;
```

```
int *q = a[0];
```

Dvojrozměrné pole a ukazatele

- A ještě jednou se podíváme na deklaraci pole *a* a proměnných *p* a *q*:

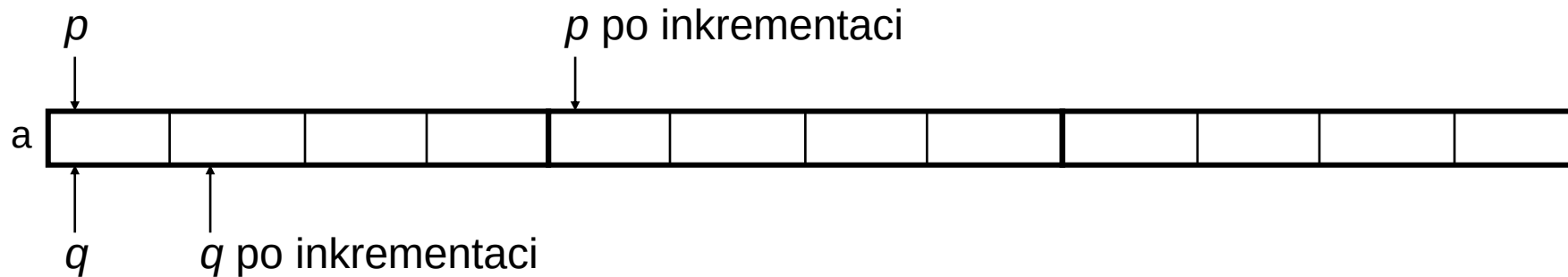
```
const int M = 3, N = 4;
```

```
int a[M][N];
```

```
int (*p)[4] = a;
```

```
int *q = a[0];
```

- Kam ukazují proměnné *p* a *q*?



- Kam ukazují proměnné *p* a *q* po provedení příkazů

```
p++;
```

```
q++;
```

Dvojrozměrné pole a ukazatele

- Průchod dvojrozměrným polem pomocí ukazatelů ilustruje následující program

```
int main() {
    const int M = 3, N = 4;
    int a[M][N];
    for (int i=0; i<M; i++)
        for(int j=0; j<N; j++)
            a[i][j] = 10*i+j;
    int (*p)[N];      /* ukazatel na řádek */
    int *q;           /* ukazatel na prvek */
    for (p=a; p<a+M; p++) {
        for(q=*p; q<*p+N; q++)
            cout << setw(5) << *q << ' ';
        cout << endl;
    }
    system("PAUSE");
    return 0;
}
```

Dvojměrné pole

- Dvojměrné pole lze vytvořit **dynamicky operátorem *new***, počet sloupců ale musí být dán konstantou
- Příklad:

```
int main() {  
    int M;  
    const int N = 4;  /* počet sloupců */  
    cout << "zadej pocet radku: ";  
    cin >> M;  
    int (*a)[N] = new int[M][N];  /* dynamická deklarace */  
    for (int i=0; i<M; i++)  
        for(int j=0; j<N; j++)  
            a[i][j] = 10*i+j;  
    ...  
}
```

- Toto omezení vyplývá z toho, že v C++ (podle ANSI) musí být v době překladač známa velikost vnitřní reprezentace každého typu

Dvojmrozměrné pole

- Dvojmrozměrné pole, jehož počet řádků i počet sloupců lze zadat proměnnou, můžeme realizovat pomocí objektu s touto strukturou:



- Pro objekt přetížíme operátor [], který vrátí ukazatel na začátek řádku s indexem i
- Takto definovaný operátor umožní, abychom na jeho výsledek znovu použili operaci indexování, tj. aby byl dovolen zápis
`a[i][j]`
- Počet řádků a počet sloupců budou dány parametry konstruktoru, který zařídí dynamickou alokaci pole
- Pro třídy objektů dynamického dvojmrozměrného pole zavedeme šablonu

Šablona třídy pro dynamické dvojrozměrné pole

```
template <class T>
class Matice {
    int radku, sloupcu;
    T *p;
public:
    Matice(int r, int s){ /* konstruktor matice */
        radku = r; sloupcu = s;
        p = new T[radku*sloupcu];
    }
    ~Matice() { /* destruktork matice */
        delete [] p;
    }
    int pocetRadku() {return radku;}
    int pocetSloupcu() {return sloupcu;}
    T* operator[](int i) { /* přetížení operátoru [] */
        return p + (i*sloupcu);
    }
};
```

Příklad použití šablony

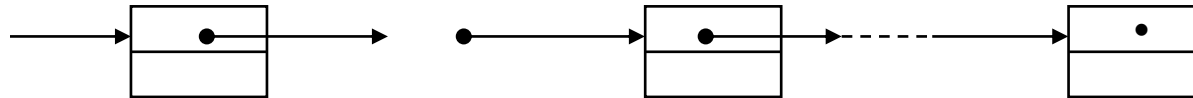
```
#include "matice.h"

int main() {
    int m, n;
    cout << "zadej pocet radku a pocet sloupce: ";
    cin >> m >> n;
    Matice<int> a(m,n); /* deklarace matice */
    for (int i=0; i<a.pocetRadku(); i++)
        for(int j=0; j<a.pocetSloupce(); j++)
            a[i][j] = 10*i+j;
    for (int i=0; i<a.pocetRadku(); i++) {
        for(int j=0; j<a.pocetSloupce(); j++)
            cout << setw(5) << a[i][j] << ' ';
        cout << endl;
    }
    system("PAUSE");
    return 0;
}
```

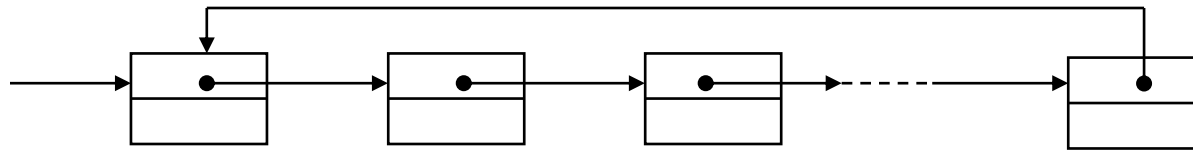
Spojový seznam

- Spojový seznam (linked list) je posloupnost prvků, kde každý prvek obsahuje spoj (link) na následující prvek.

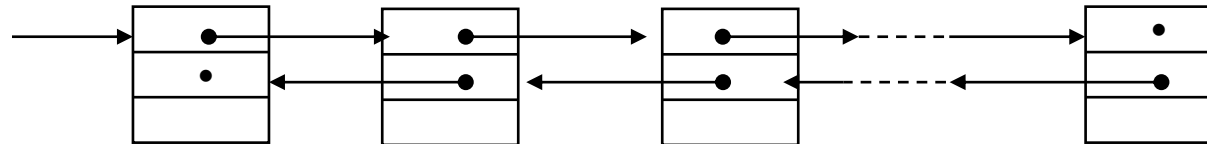
- Jednosměrný spojový seznam:



- Cyklický spojový seznam:



- Dvojsměrný spojový seznam:



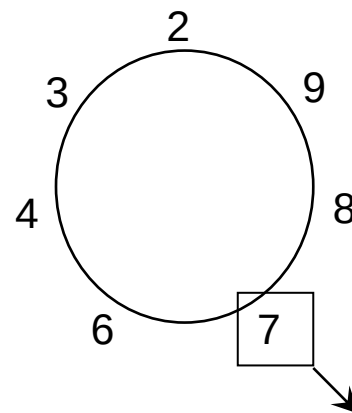
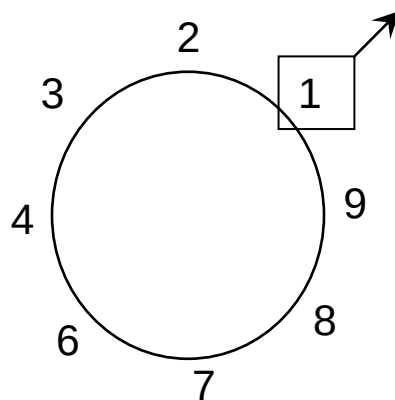
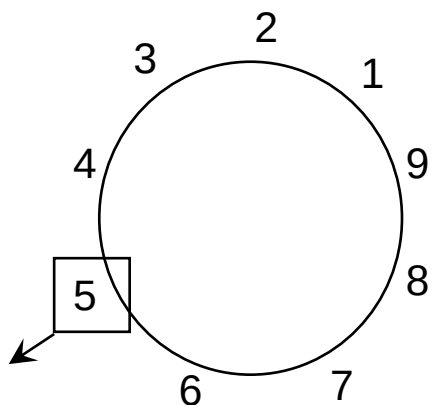
- Prvky spojového seznamu nejčastěji vytváříme dynamicky, spoje jsou ukazatele

Spojový seznam

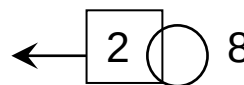
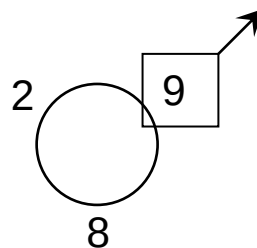
- Výhody spojového seznamu
 - snadno lze přidávat nebo vkládat nové prvky
 - snadno lze prvky vypouštět
 - není třeba stanovit omezení na počet prvků
- Nevýhody spojového seznamu
 - přístup k libovolnému prvku nemá konstantní složitost, ale lineární
 - spojový seznam obsahující n prvků spotřebuje více paměti, než pole s n prvky
- Spojový seznam lze zavést jako abstraktní datový typ s operacemi jako je vložení prvku, vypuštění prvku atd. To uděláme později.
- V řadě úloh stačí pro prvky spojového seznamu zavést vhodný datový typ (strukturu nebo třídu) a manipulace se seznamem realizovat přímo
- Jako příklad přímé manipulace se spojovým seznamem (bude cyklický) ukážeme řešení tzv. *Josefusova problému*

Josephusův problém

- Zadání:
 - Na obvodu kruhu je N očíslovaných objektů. Postupně přes objekty procházíme, přičemž každý M -tý odstraníme. Který objekt zůstane poslední?
- Příklad pro $N = 9$ a $M = 5$



...



poslední
zůstane 8

Josephusův problém

- Proč se tento problém označuje jako Josephusův?
- Odpověď z historie:
 - Flavius Josephus byl židovský historik žijící v 1. století, který se zúčastnil války židů proti Římanům. Stalo se, že on a jeho 40 spolubojovníků se ocitlo v pasti v jeskyni obklíčení Římany. Místo zajetí zvolili smrt: postupně každý druhý se zabije. Nakonec zbyl Josephus a ještě jeden, kterého Josephus přesvědčil, aby se vzdali. Bylo to štěstí, prozřetelnost nebo si vypočítal vhodné pořadí??
- Více najdete např. na adrese http://en.wikipedia.org/wiki/Flavius_Josephus
- Josephusův problém lze řešit analyticky
- My jej vyřešíme pomocí cyklického spojového seznamu, ze kterého budeme postupně odstraňovat prvky

Josephusův problém

- Spojový seznam bude tvořen objekty typu *Node*:

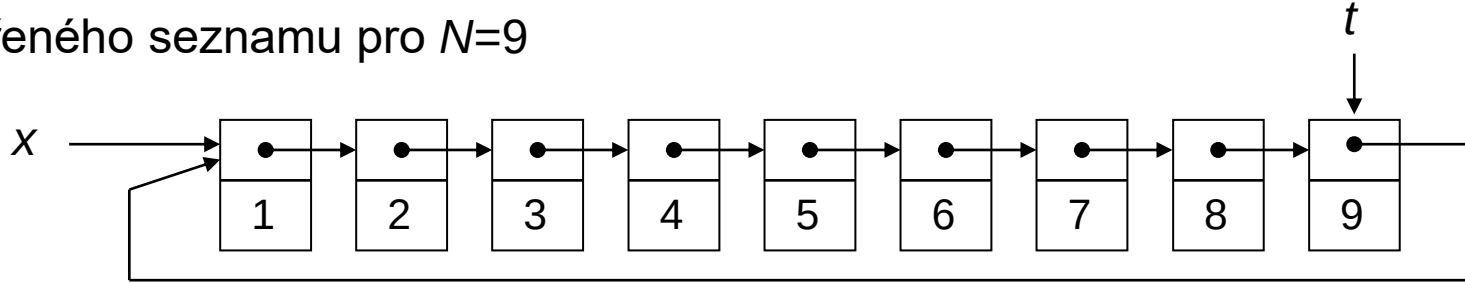
```
struct Node {  
    int id;  
    Node *next;  
    Node(int i) : id(i) {}  
};
```

- V programu přečteme *N* (počet objektů) a *M* (modul pro vypouštění) a vytvoříme cyklický seznam

```
int main() {  
    int i, N, M;  
    Node *t, *x;  
    cin >> N >> M;  
    // vytvoření cyklického spojového seznamu  
    t = new Node(1); x = t;  
    for (i=2; i<=N; i++) {  
        t->next = new Node(i);  
        t = t->next;  
    }  
    t->next = x;    /* zacyklení seznamu */  
}
```

Josephusův problém

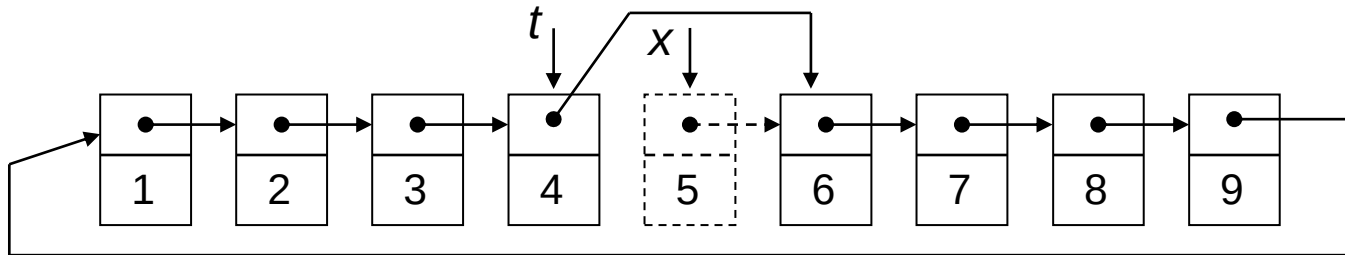
- Příklad vytvořeného seznamu pro $N=9$



- Potom, dokud v seznamu nezbude pouze jeden objekt, budeme seznamem procházet a vypouštět každý M -tý objekt

```
while (t != t->next) {  
    for (i=1; i<M; i++) t = t->next;  
    x = t->next; t->next = x->next;  
    delete x;  
}
```

- Příklad seznamu po prvním vypuštění ($M=5$):



Josephusův problém

- Výsledný program:

```
int main() {
    int i, N, M;
    Node *t, *x;
    cout << "Josephusuv problem" << endl;
    cout << "zadej N a M: ";
    cin >> N >> M;
    t = new Node(1); x = t;    /* vytvoření seznamu */
    for (i=2; i<=N; i++) {
        t->next = new Node(i);
        t = t->next;
    }
    t->next = x;
    while (t != t->next) {
        for (i=1; i<M; i++) t = t->next; /* vypuštění M-tého */
        cout << t->next->id << ' ';
        x = t->next; t->next = x->next;
        delete x;
    }
    cout << t->id << endl;
    system("PAUSE");
    return 0;
}
```

Pole ~ spojový seznam

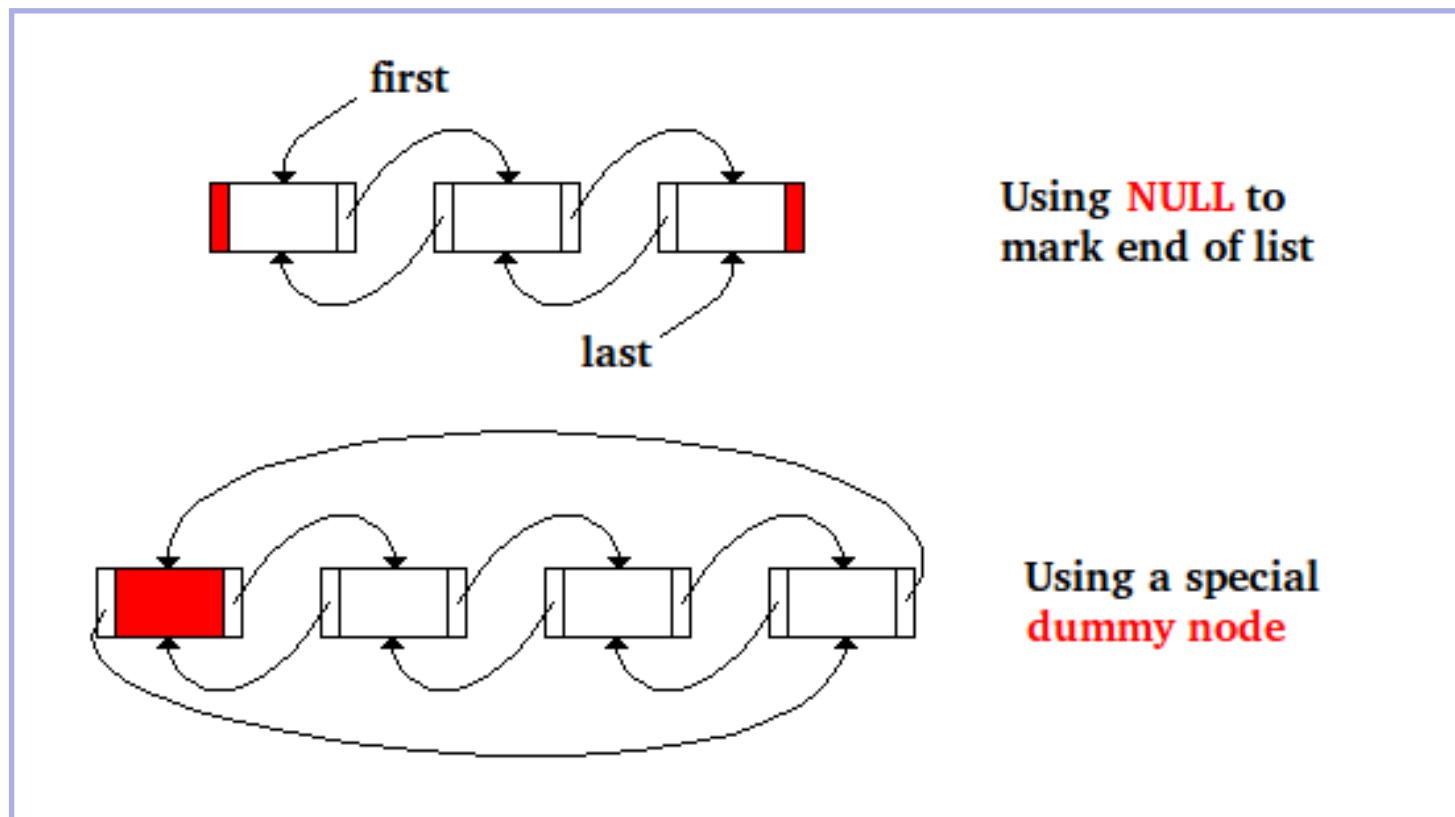
- Porovnejme řešení Josefusova problému s Eratosthenovým sítím
 - v obou případech se opakovaně vypouštějí prvky z uspořádané množiny přirozených čísel
 - v případě ES se vypuštění děje změnou hodnoty prvku pole (index udává číslo, hodnota říká, zda dané číslo je v množině), počet prvků pole se nemění
 - to umožňuje stále konstantní složitost přístupu k prvku pole reprezentujícího množinu při vypouštění násobků dalších čísel
 - v případě JP se odstraňuje prvek množiny ze spojového seznamu a počet prvků seznamu se zmenšuje
 - to umožňuje stále konstantní složitost při vypouštění dalšího M -tého prvku modifikované množiny
- Zkuste řešit ES pomocí spojového seznamu a JP pomocí pole; řešení bude mnohem složitější
- Závěr: Při volbě datové struktury (pole nebo spojový seznam) je třeba vyjít z charakteru operací, které se budou provádět

Abstraktní datové typy

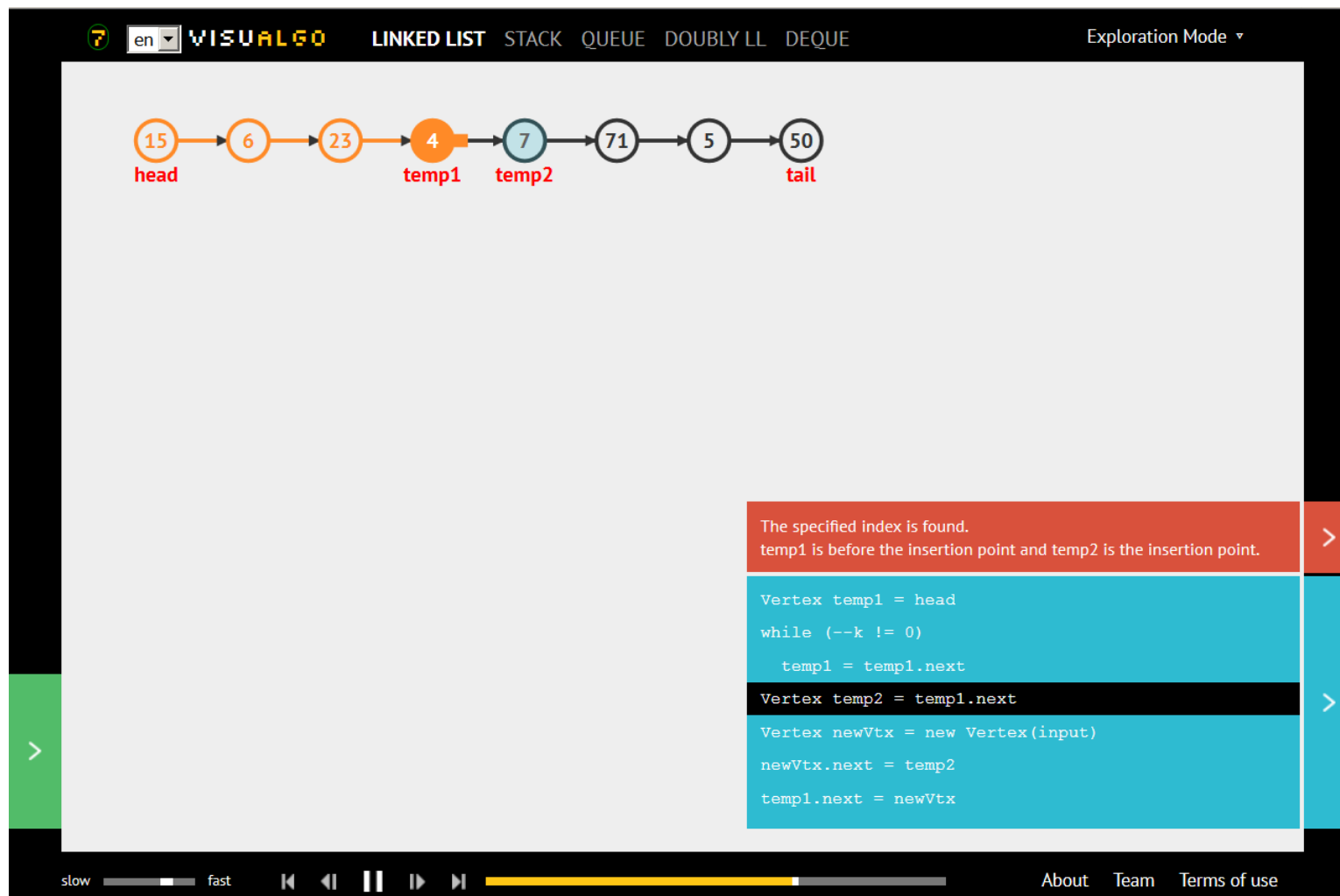
- Specifikace (abstraktního) datového typu
- Implementace datového typu
- Sekvenční kontejnery – různě organizované sekvence
 - Zásobník (Stack)
 - Fronta (Queue)
 - Seznam (List)
 - Pole (Array, Vector, Matrix, ...) umožňuje náhodný přístup k prvkům
- Asociativní kontejnery – pro vkládání, odebrání a vyhledávání je důležitý klíč prvku
 - Množina (Set)
 - Tabulka (Table, Map)

Speciální uzel (SENTINEL) ve spojovém seznamu

Zrychluje určité typy operací tím, že za cenu poněkud vyšší paměťové náročnosti eliminuje některé testy.



Spojový seznam, informace + vizualizace

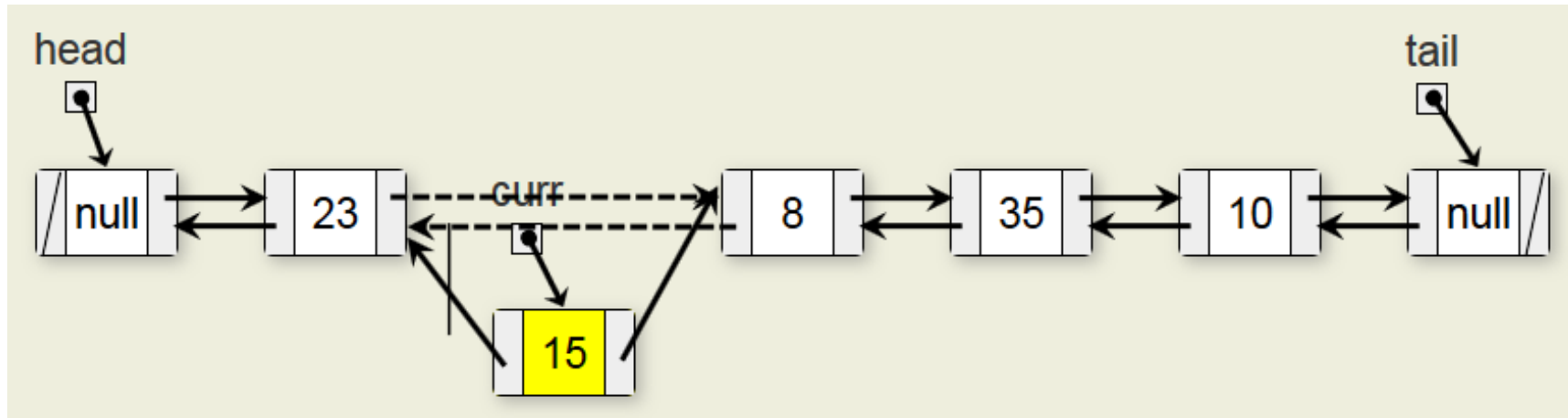


Zdroj: <https://visualgo.net/en/list>

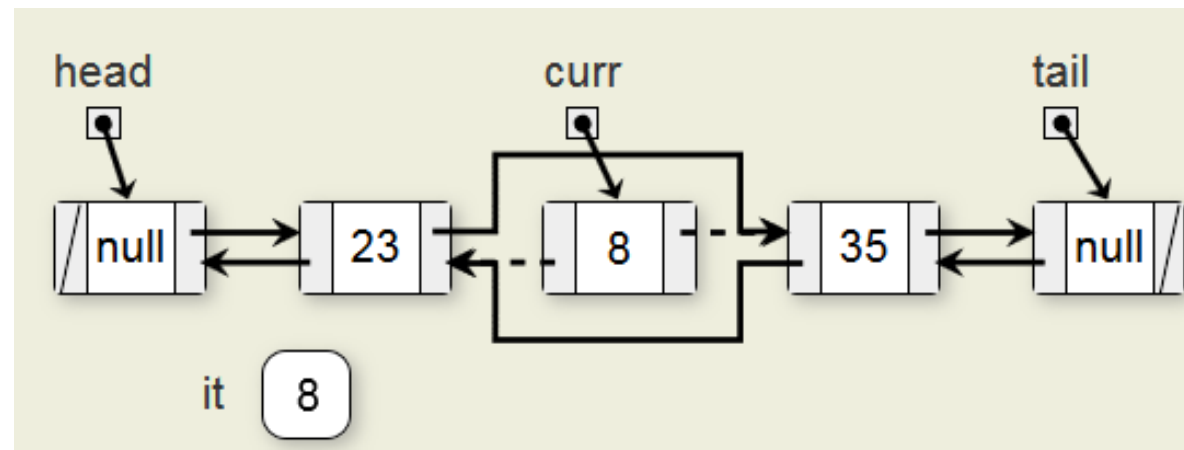
Spojový seznam, animace

(bohužel, C++ kód se odsud časem vytratil)

Přidání



Odstranění



Zdroj <https://opensa-server.cs.vt.edu/ODSA/Books/Everything/html/ListLinked.html>

Jednosměrný spojový seznam

1/3

```
#include <iostream>
using namespace std;
```

```
struct Node {
    int data;
    Node* next;
};
```

```
int main()
{
    Node* head;
    head = NULL;
```

```
    Node* temp = new Node();
    temp->data = 2;
    temp->next = NULL;
```

```
    head = temp;
```

Poznámka:

Komentovaná audiovizuální verze následujících dvou příkladů je k dispozici v přednáškových poznámkách z loňského roku, dostupných v MS Teams DSA 2021

Screenshot(y) z [AP4](#)

Jednosměrný spojový seznam

1/3

```
#include <iostream>
using namespace std;
```

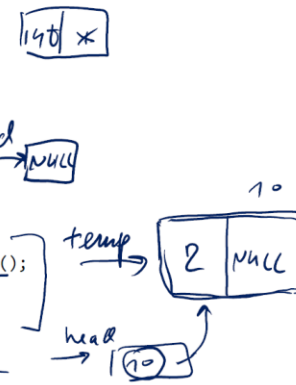
```
struct Node {
    int data;
    Node* next;
};
```

```
int main()
{
    Node* head;
```

```
    head = NULL;

    Node* temp = new Node();
    temp->data = 2;
    temp->next = NULL;
```

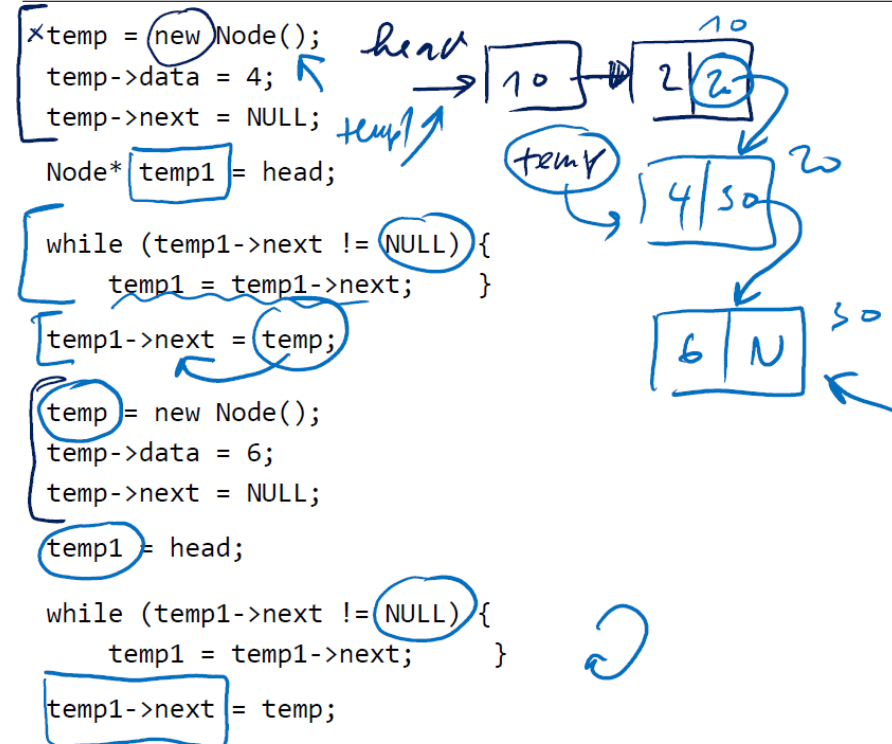
```
    head = temp;
```



```
temp = new Node();  
temp->data = 4;  
temp->next = NULL;  
  
Node* temp1 = head;  
  
while (temp1->next != NULL) {  
    temp1 = temp1->next;  
}  
  
temp1->next = temp;  
  
temp = new Node();  
temp->data = 6;  
temp->next = NULL;  
  
temp1 = head;  
  
while (temp1->next != NULL) {  
    temp1 = temp1->next;  
}  
  
temp1->next = temp;
```

Jednosměrný spojový seznam

2/3



13

```
// vypis
temp1 = head;
cout << temp1->data;
while (temp1->next != NULL) {
    temp1 = temp1->next;
    cout << temp1->data;
}

return 0;
}
```

Jednosměrný spojový seznam

3/3

```
// vypis
temp1 = head;
cout << temp1->data;
while (temp1->next != NULL) {
    temp1 = temp1->next;
    cout << temp1->data;
}

return 0;
}
```

head → list

↓ ↓ ↓
2 4 6

Mějme obousměrně vázaný spojový seznam přirozených čísel (ukazatele *next* a *prev*), který si udržuje informace o svém začátku, konci a aktuální pozici (ukazatele *begin*, *tail*, *mark*). Za předpokladu:

```
Node(T x, Node *n) {val = x; next = n; prev = n->prev;}
```

doplňte následující kódové fragmenty pro:

1. Vložení prvku před kurzor

```
void List::insert(T x) {  
    Node *p = new Node(x, mark);  
    ???;  
    if (atBegin())  
        ???;  
    else  
        ???;  
}
```

Obousměrný spojový seznam

1/2

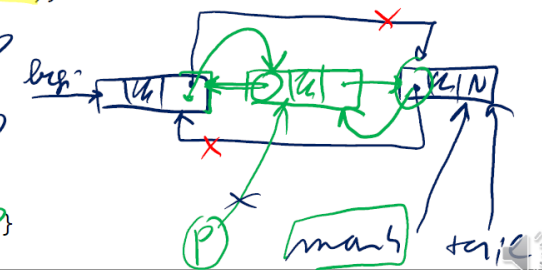
Mějme **obousměrně vázaný** spojový seznam **přirozených** čísel (ukazatele **next** a **prev**), který si udržuje informace o svém začátku, konci a aktuální pozici (ukazatele *begin*, *tail*, **mark**). Za předpokladu:

```
Node(T x, Node *n) {val = x; next = n; prev = n->prev;}
```

doplňte následující kódové fragmenty pro:

1. Vložení prvku před kurzor

```
void List::insert(T x) {  
    Node *p = (new Node(x, mark));  
    ???; mark->prev = p;  
    if (atBegin())  
        ???; begin = p;  
    else  
        ???; p->prev->next = p;  
}
```



15

2. Odebrání prvku pod kurzorem

```
void List::remove() {  
    if (atEnd()) return;  
    Node *p = mark;  
  
    ???;  
  
    if (atBegin())  
  
        ???;  
  
    else  
  
        ???;  
  
    ???;  
  
    delete p; }  
}
```

Obousměrný spojový seznam

2/2

2. Odebrání prvku pod kurzorem

```
void List::remove() {  
    if (atEnd()) return;  
    Node *p = mark;  
    ???;  
    if (atBegin())  
        ???;  
    else  
        ???;  
    delete p; }  
}
```

